

Your Index Is Fine, Your Query Encoder Is Not: Query Distillation over Frozen Indexes

Dylan Couzon Evgeniya Sukhodolskaya
dylan.couzon@qdrant.com jenny@qdrant.com
Kumar Shivendu Andrey Vasnetsov
kumar.shivendu@qdrant.com andrey@qdrant.com
Qdrant

October 2026

Abstract

Document embeddings are reused across searches, amortizing document-encoding cost, while every uncached query pays for encoding. Choosing a cheaper query encoder independently of the stored document representation could reduce that recurring cost and let requests use different compute budgets over one index.

We fit query replacements across 26 embedding spaces: a table that averages learned token vectors and a frozen transformer with a learned projection. The strongest original query path yields the weakest registered table student. Original quality alone scarcely ranks the students; after accounting for embedding dimensionality, the expected positive association returns, while higher dimensionality predicts weaker students. Cheap query vectors also change search: table students need median graph-effort multipliers of two to four to recover the same fraction of their own exact neighbors. The deficit recurs in a jointly trained lookup model.

Constella demonstrates the payoff over the frozen document index of Stella, a pretrained English embedding model. Its compact transformer Nano and token table Zero retain 90.5% and 81.4% of Stella’s exact-search nDCG@10 on BEIR-15, with separately measured CPU encoding medians of 2.25 ms and 0.044 ms. Nano’s final optimization took 57.3 A100 hours. This one-time construction can be amortized over recurring savings when retained relevance suffices, while the shared index makes query compute selectable per request.

1. Introduction

Query and document encoders serve different computational roles. Document embeddings are stored and reused across searches, allowing an expensive document representation to be amortized. Every uncached query incurs its own encoding cost. This asymmetry raises the question of choosing the query encoder independently of the stored document representation. Query-side distillation builds a cheaper encoder whose vectors remain compatible with the documents [QED, EmbedDistill, LEAF], separating the representation retained in the index from the computation used to search it.

Constella makes that choice concrete. It consists of two query encoders aligned to the 1024-dimensional document space of **Stella**, the pretrained English model `stella_en_400M_v5` [Stella]. **Nano** is a separately trained 34.5-million-parameter transformer, and **Zero** is a trained token-vector table. Together with Stella’s original query path, they provide three query-compute budgets over identical document vectors. A request can select an encoder without changing the document representation.

We study what this separation gives and what it costs through two empirical questions and a demonstrated capability. First, what exact-search quality can a linearly fitted query student recover from a frozen space, and which properties predict it? Second, how much additional graph-search effort does query replacement

require, and can that penalty be predicted? The cross-space study fits inexpensive students, evaluates their relevance by exact search, then searches their queries over unchanged graphs. Constella combines the resulting considerations with separately trained encoders: their construction costs, recurring encoding and search costs, and the operating choices available when the compute budget can change per request.

The strongest original query path need not yield the strongest cheap student. In our registered comparison, gte-large-en-v1.5 has the highest original retrieval score but the lowest fitted-table score. Across the broader roster, accounting for dimensionality reveals the positive quality association hidden in the pooled comparison. That improves prediction across held-out families, although directly measuring student development retrieval makes a better original-roster selection. Query replacement also changes graph search: token-table queries recover fewer exact neighbors at fixed effort in every measured space on two of three corpora, and the deficit recurs in a lookup path trained jointly with its document tower. Encoding speed alone therefore leaves both a relevance question and a search-cost question unanswered.

Keeping the document vectors preserves an asset that would otherwise need replacement. Changing document encoders requires corpus re-embedding and index rebuilding; maintaining availability during migration can require parallel indexes and ingestion pipelines, extra infrastructure load, relevance validation, and rollback [Index migration]. The shared-index family places the choice at the query boundary. Its value comes from selectable computation and retained relevance together, with each path’s search requirements contributing to its operating cost.

2. Related work

Query-side distillation over a frozen document encoder is established. QED aligns a compact query encoder with an existing dense retriever [QED]; EmbedDistill studies geometric distillation against frozen document representations [EmbedDistill]; and LEAF provides teacher-aligned compact representations [LEAF]. pyNIFE fits a static token table against a frozen teacher [pyNIFE]. LightRetriever trains a lookup query path jointly with a document tower [LightRetriever]. Our contribution is a common-recipe comparison across spaces and a study of the search consequences of substituting queries over unchanged graphs.

Stronger teachers can produce weaker students in general distillation [Cho and Hariharan 2019] and dense retrieval [PROD]. Those studies discuss capacity gaps between teacher and student. We test embedding dimensionality as a predictive property across spaces, alongside the original query path’s measured relevance. The conditional model, family-held-out predictions, and prospective test distinguish this question from choosing an encoder by its benchmark score alone.

Graph-based approximate nearest-neighbor search is sensitive to query placement relative to indexed vectors [HNSW]. Out-of-distribution queries can require more work, and query-aware graph construction can mitigate difficult query distributions [OOD-DiskANN, RoarGraph]. Here the graph stays fixed while the query encoder changes. We measure both recovery of each path’s exact neighbors and the resulting relevance loss. The distinction matters because missing an exact neighbor need not remove a relevant result.

Per-query retrieval-strategy selection and query performance prediction also have substantial prior work [Arabzadeh et al., QPP]. A compatible family permits selection, but compatibility alone supplies no rule for allocating compute according to relevance. Our routing results separate that prediction problem from an operating policy based on request context.

3. Experimental setup

3.1 Spaces, students, and data

The cross-space roster contains 26 public English embedding encoders sharing the 30,522-entry BERT Word-Piece vocabulary. This requirement belongs to the table comparison; the contextual head fits vectors from a separate feature encoder. Ten spaces were registered before results, and 16 were added as an exploratory expansion. The roster spans nine related model families and embedding dimensionalities of 384, 768, and 1024. Each original query path searches its own frozen document vectors. Appendix C retains the complete scores and model identities.

We fit two query-student representations by ridge regression to each original query path’s vectors. The **table student** assigns a vector to each token, mean-pools the token occurrences, and L2-normalizes the result. Its regression penalty anchors the rows to vectors obtained by encoding individual tokens through the original model. The **head student** uses contextual features from a frozen bge-small transformer [BGE]: attention-masked mean states from layers 12, 8, and 4 are concatenated into 1,152 features. A fitted projection with bias maps those features into the document space, followed by normalization. Only the projection is learned, so construction is inexpensive while inference still runs the backbone.

Both students use the same cleaned 337,981 fit queries and each original model’s published query prompt for targets. Penalties are selected by exact retrieval on the CQADupStack physics and programmers development forums, then frozen before public scoring. Quality is normalized discounted cumulative gain at rank 10, **nDCG@10**, under exact search on NFCorpus, SCIDOCs, SciFact, TREC-COVID, FiQA, and ArguAna [BEIR]. The primary outcome is the unweighted mean over these six datasets. The first four, without disclosed Stella exposure, form a sensitivity outcome.

The fitted students provide a controlled construction comparison, while released Zero and Nano use additional optimization and different training pools. Zero learns token vectors and scalar weights through vector alignment, teacher-score matching, and contrastive ranking against frozen Stella document vectors. Its served table uses square-root token-count weights and int8 rows. Nano starts from bge-small, concatenates the same three layers, and learns a projection into Stella’s space; backbone and projection are then trained jointly against Stella targets. Training alternates three query batches with one document batch. Stella remains the document encoder for retrieval.

These recipe differences matter when connecting the cross-space findings to Constella. The fitted head holds its backbone fixed; Nano updates it. The fitted table uses ordinary mean pooling; Zero adds ranking optimization and a different served pooling rule. We report their results as separate experiments, with reproduction details in Appendix B.

3.2 Search and evidence status

Approximate search uses Qdrant 1.19.1, a vector search engine, with uncompressed vectors, cosine distance, and hierarchical navigable small-world (**HNSW**) graphs [Qdrant 1.19.1, HNSW]. The cross-space workloads are FiQA, SCIDOCs, and TREC-COVID. Two builds per space differ in seeded document insertion order. The graph parameters stay fixed while each original query path and its students search with varying **ef**, the parameter controlling graph-search breadth.

We measure **relevance loss**, the fractional reduction from a path’s own exact nDCG@10, and **exact-neighbor recovery**, the fraction of its exact top-10 recovered by graph search. Each path has its own reference. Effort multipliers compare the smallest student **ef** reaching the original query path’s loss or recovery at **ef**=64, divided by 64 and averaged over builds. A target not reached by the largest setting is censored, with a multiplier of at least eight. Appendix B specifies ties, parity checks, and exclusions.

“Registered” identifies evaluations whose decision rules preceded observations. “Exploratory” identifies subsequent analyses. The cross-space quality and search screens are exploratory, with their analyses written before scoring. The conditional-quality hypothesis was developed after seeing the original 26 spaces; its prospective test committed predictions by hash before fitting new students. Constella’s registered evaluations and exploratory interpretations remain distinguished below.

4. Quality: what the document space predicts

The initial hypothesis was that stronger original retrieval would produce stronger fitted students, with the strongest original encoder yielding the strongest student. The registered comparison reverses that selection rule: gte-large-en-v1.5 scores 0.5970 but yields the weakest registered table at 0.2455; Stella scores 0.5745 and yields the strongest registered table at 0.3974. Across all 26 spaces, original and student quality have rank correlation +0.09 for both representations, with 95% checkpoint intervals of $[-0.39, +0.52]$ for the table and $[-0.36, +0.51]$ for the head.

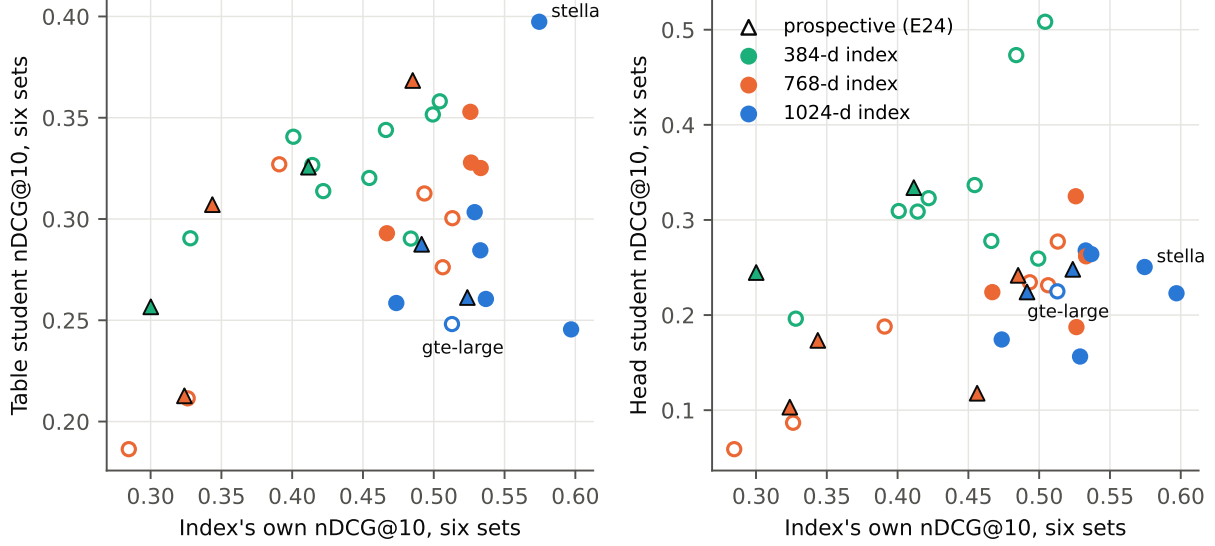


Figure 1: Student quality against the original query path’s quality for table (left) and head (right) students, colored by dimensionality. Filled points are registered spaces; triangles are prospective spaces, eight for the head and seven for the table.

The competing hypothesis conditions on dimensionality: at a given embedding size, stronger spaces should favor stronger students, while higher dimensionality should predict weaker students after accounting for original quality. We model absolute student nDCG@10 from original quality and log2 dimensionality. Standardized coefficients express changes in student-score standard deviations per predictor standard deviation. Intervals use 10,000 checkpoint resamples; held-out prediction fits on eight families and predicts the ninth.

Table 1. Conditional prediction of student six-set nDCG@10 across 26 spaces (exploratory, analysis pre-specified). Coefficients have 95% intervals; R^2 is in-sample fit, and held-out ρ is Spearman correlation across family-held-out predictions.

Student	Quality coefficient	Dimensions coefficient	R^2	Held-out ρ
Head	+0.69 [+0.40, +0.97]	−0.83 [−1.14, −0.60]	0.70	+0.69
Table	+0.63 [+0.06, +0.89]	−0.64 [−0.95, −0.32]	0.48	+0.40

Original quality and dimensionality correlate at +0.61. Their opposing associations with student quality explain why the pooled comparison obscures the positive quality signal. Quality-only models have R^2 of 0.12 for the head and 0.14 for the table, with held-out correlations −0.17 and −0.15. Adding dimensionality improves both. On the four-set sensitivity outcome, the dimensionality association persists, while the table’s original-quality coefficient interval slightly crosses zero. The result is predictive: the roster does not separate dimensionality causally from family and other model properties.

We test the forecast in two ways. A model fitted on the ten registered spaces ranks the 16 later spaces at Spearman 0.85 [0.53, 0.99] for the head and 0.70 [0.20, 0.96] for the table, against 0.37 and 0.16 using original quality alone. This validation is retrospective because the hypothesis followed observation of all 26. The prospective test instead fixed eight further encoders, committed predictions by hash, then fitted and scored their students. One table fit stopped at its convergence gate, leaving seven tables and eight heads.

The prospective predictions rank the heads at $+0.86$ $[+0.33, +1.00]$ and tables at $+0.71$ $[-0.12, +1.00]$, against $+0.26$ and $+0.39$ for original quality alone. Mean absolute score error is 0.039 for the head and 0.032 for the table. The head result supports positive prospective ranking; the table interval leaves its result uncertain. Appendix C gives every actual and predicted score.

Ranking prediction and choosing the best student are different tests: on the original roster, the conditional model’s top choices, each predicted by a fit on the other 25 spaces, lose 0.249 nDCG@10 for the head and 0.107 for the table relative to the best available students. Selecting by directly measured development retrieval loses 0.035 and 0.000; on the prospective candidates, both rules select the best head and a table 0.043 below the best.

Direct screening is practical because table fits took four to seven minutes per registered space on an A100, including query encoding except for cached Stella targets. It also tests the relevant outcome: two e5 tables match their original query vectors at mean cosine 0.90 and 0.89, against Stella’s 0.78, yet retrieve worse. Better vector agreement does not certify better ranking against the stored documents. The inexpensive fit therefore supplies both a forecastable object and a direct retrieval test, with the latter making the better original-roster selection.

5. Search cost: what query replacement changes

Exact retrieval establishes the student representation’s quality. Approximate search adds another possible loss over the same document vectors. We hypothesized that replacing the original queries would require more graph effort to recover the same fraction of each path’s exact neighbors. This exploratory screen holds the graphs fixed between query paths and measures 25 spaces after the parity exclusion described in Appendix B.

At `ef=64`, table students recover fewer exact neighbors in all 25 spaces on FiQA and TREC-COVID, and in 17 of 25 on SCIDOCS. Mean student-minus-original gaps are -2.9 , -3.9 , and -0.8 percentage points, respectively. Median recovery-effort multipliers are four on FiQA and TREC-COVID and two on SCIDOCS. The head students also have deficits in 24 of 25 spaces on FiQA and 23 of 25 on TREC-COVID. Query substitution affects graph search beyond the lookup representation alone.

Relevance gives a less uniform answer. Matching the original query path’s relative nDCG loss requires greater effort in all 25 FiQA spaces, with a median multiplier of four among reached targets and four spaces censored in at least one build. The multiplier exceeds one in 17 of 25 SCIDOCS spaces but only 12 of 25 TREC-COVID spaces. On TREC-COVID’s 50 queries, the original query path loses only 0.2% of nDCG at the reference setting, making its threshold sensitive to very small differences. The recurring neighbor-recovery deficit therefore has corpus-dependent relevance consequences.

Corpus size contributes to the variation. An exploratory, pre-specified control subsamples FiQA to SCIDOCS’s 25,657 documents while retaining judged positives and rebuilding two graphs per space. The recovery gap moves from -2.9 to -1.2 points, a paired change of $+1.8$ $[+1.3, +2.3]$ across spaces. It remains 0.4 points larger than SCIDOCS’s, with interval $[-0.9, -0.1]$ for the signed difference. Smaller corpus size reproduces much of the attenuation, with a remaining corpus difference.

Query placement also supplies a predictor. Before computing features, we specified the ratio of query-to-nearest-document distance to typical nearest-neighbor distance among documents, alongside other geometry measurements. Across 75 space-by-workload points, the ratio from original queries correlates with the recovery gap at -0.64 $[-0.75, -0.48]$. Its family-held-out predictions correlate at $+0.64$ with mean absolute error 1.26 percentage points, versus 2.22 for the training-fold mean. Spaces whose original queries sit farther from documents tend to incur larger replacement deficits.

The original-query ratio marks susceptibility, and its cause stays open: the student-minus-original change predicts poorly on held-out families. Transfer across workloads is weaker than transfer across families: the model with all features and covariates, trained on two workloads, ranks the third at $+0.50$, but its error is 2.55 points against 2.33 for the fold-mean baseline. This predictor helps rank susceptible spaces within the measured setting; the penalty’s magnitude still depends on the workload.

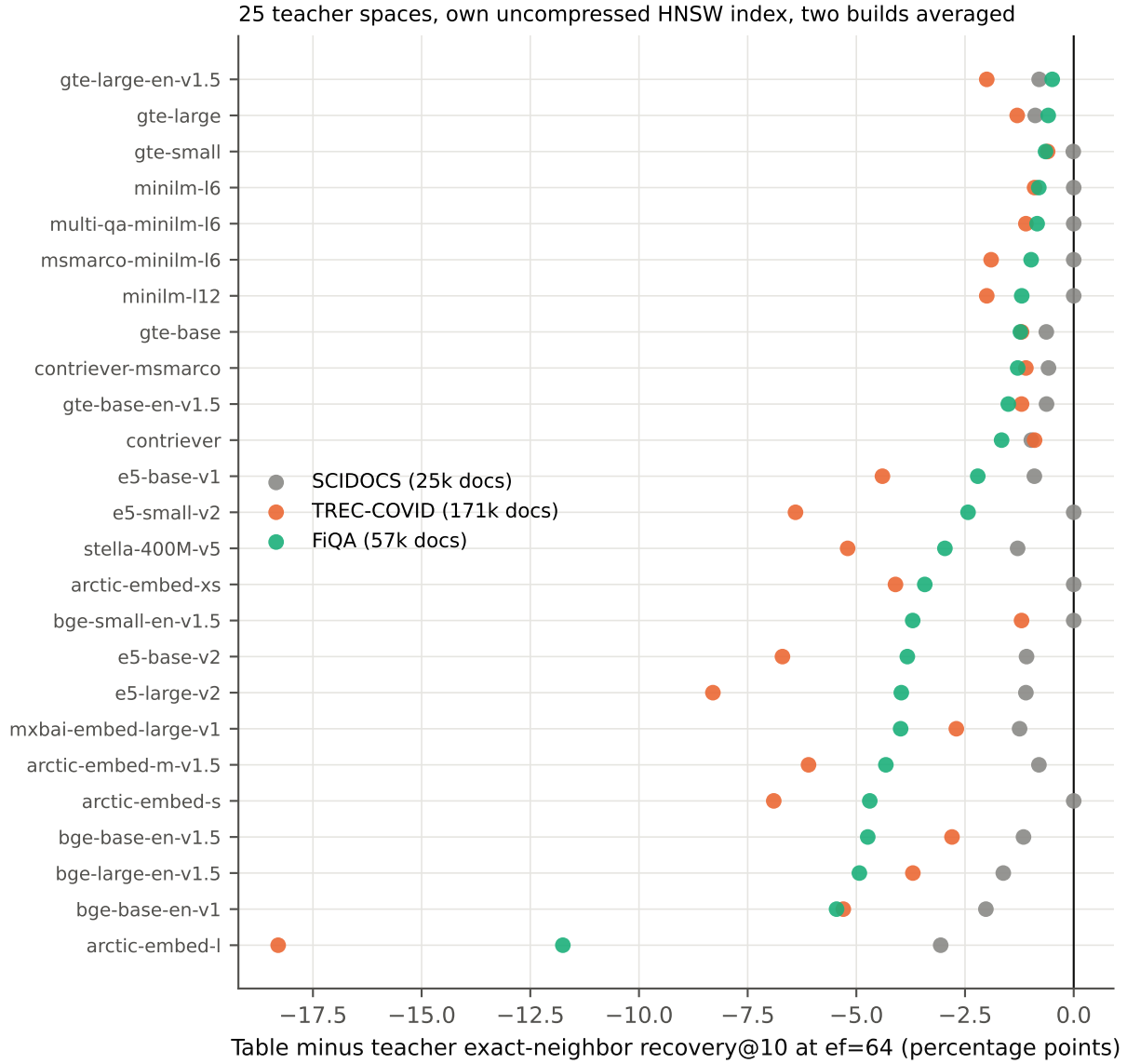


Figure 2: Student-minus-original exact-neighbor recovery at $ef=64$, by space and workload, averaged over two builds. Negative values mean the table student recovers fewer of its own exact neighbors.

We then test recurrence beyond students fitted to a frozen teacher. In an exploratory experiment specified before encoding, LightRetriever’s released Qwen2.5-1.5B model supplies both lookup and full query paths over its own document vectors [LightRetriever]. Under its primary web-search prompt, lookup-minus-full recovery gaps are -3.6 $[-4.3, -2.9]$ points on FiQA and -1.7 $[-2.1, -1.4]$ on SCIDOCS. Matching full-path recovery requires four times the **ef** on both corpora. Relative relevance-loss effort agrees on FiQA; on SCIDOCS, both paths lose under 0.1% of nDCG at the reference, making that comparison sensitive to small changes.

The deficit thus recurs in a jointly trained lookup model as well as our closed-form students. Search-effort multipliers measure graph breadth. Per-path settings recover the measured targets where the sweep reaches them; relevance and recovery remain distinct criteria for that adjustment. Section 6 measures how the additional work changes the operating choices within Constella.

6. One index, several query budgets

Constella turns query-side separation into a family: Stella, Nano, and Zero search the same populated document collections, while providing different relevance and compute budgets. The original query path remains available alongside its replacements. The family preserves the stored representation across these choices, so the operating policy can choose an encoder for each request.

The registered descriptive BEIR-15 evaluation measures exact quality. Encoding is measured separately on an Apple M5 Pro CPU with four threads, batch one, and 100 real queries of five to 12 words. Table 2 places these two measurements side by side; its asset sizes describe encoder files, not resident memory.

Table 2. Constella over one frozen Stella document representation. Quality is exact BEIR-15 macro nDCG@10; retention is the ratio to Stella. Encoding medians follow the registered medium-query CPU protocol.

Query encoder	nDCG@10	Retention	Encode p50	Asset size
Stella	0.5614	reference	31.6 ms	1669.6 MiB
Nano	0.5081	90.5%	2.25 ms	132.3 MiB
Zero	0.4572	81.4%	0.044 ms	90.1 MiB

Construction and recurring cost. Nano’s final optimization took a measured 57.3 A100 hours, about \$95 at the recorded rental rate. Zero’s retraining was historically estimated at 20 minutes with prepared targets, plus 8 to 12 hours of target encoding for a new space. Preparation, data generation, recipe search, failed runs, evaluation, and engineering add to the full build. The four-to-seven-minute fitted screens answer a cheaper construction question than producing these optimized models. Once constructed, a compatible query encoder can amortize that one-time cost through recurring savings, provided its retained relevance meets the workload’s requirements.

Search narrows the difference between the cheap paths. On the one-million-passage MS MARCO diagnostic, with 6,980 development queries, uncompressed search reaches within 1% of each path’s own exact nDCG at **ef**=128 for Stella, 256 for Nano, and 512 for Zero. The diagnostic preserves all judged positives and samples other passages, reducing competition relative to the full corpus. Its search measurements and Table 2’s encoding sample describe separate query populations.

Summing these component medians and extrapolating to one million queries gives Nano and Zero time shares of 13% and 11% of Stella’s, respectively. Large savings versus the original query path remain, but Zero’s additional search largely consumes its encoding advantage over Nano. At a fixed **ef** from 32 to 512, search medians differ by at most 7% across paths; the added work comes from the larger settings required to reach their respective targets. These component-time shares measure neither sustained throughput nor equal absolute relevance.

The frontier depends on the collection configuration as well as the encoder. In an exploratory selection on one binary-quantized collection of the million-passage diagnostic, using that diagnostic’s own encoding

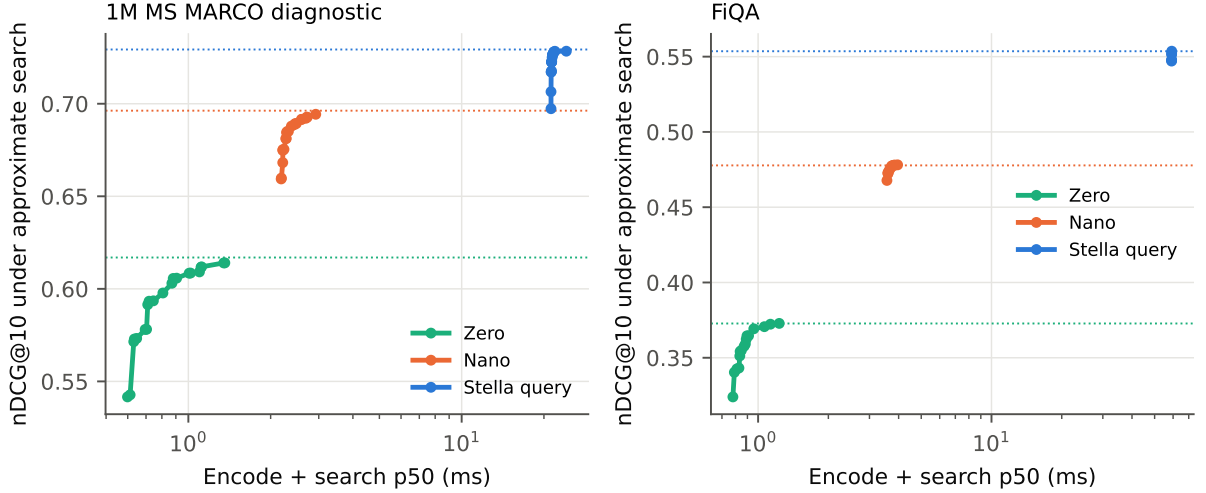


Figure 3: Approximate-search quality against encoding plus search time on FiQA and the positive-preserving MS MARCO diagnostic, registered sweep. Dotted lines are each path’s exact score.

timings and the fastest settings within 1% of each path’s exact score, Zero’s 63-fold encoding advantage over Nano becomes 2.2-fold after search, at different quality levels. Figure 3 retains the relevance coordinate that a speed ratio omits. It also shows why an encoder’s own exact score is the appropriate reference for approximation loss, while the absolute score remains necessary to compare operating choices.

A request-context policy can select among these choices. In an e-commerce example, Zero could serve searches while a user types, Nano after typing stops, and Stella on submission, with a path-specific `ef` accompanying each selection. This is an illustrative policy enabled by compatibility, not a measured typeahead workload. The same document vectors remain searchable throughout; the policy assigns computation by interaction stage, so it needs no prediction of which encoder will retrieve better for a particular query.

Automatic relevance-based allocation is a separate problem. In the registered routing analysis over 12 datasets, a label-aware oracle selecting Zero or Nano scores 0.5473 against 0.5161 for always-Nano. Yet query length, subwords per word, and pooled-vector norm improve at most 0.005 over random routing at matched Nano share. The family exposes useful headroom, but these simple features do not recover it. Request-context selection uses an available operating signal; the oracle requires relevance information unavailable at request time.

A lexical branch supplies another compatible route. Descriptive BEIR-15 fusion with BM25 raises Zero to 0.4933, or 87.9% of Stella, but its sign depends on workload, with Nano falling on MS MARCO. This route adds a sparse index and branch search, using distribution-based score fusion over 100 candidates per branch [DBSF].

7. Discussion

A frozen document representation can support several query-compute budgets, but its original retrieval score and search setting do not determine the usefulness of a replacement encoder. The quality study identifies conditional predictors and the value of direct student evaluation; the search study establishes an additional, workload-dependent recovery cost. Constella combines those constraints into a family whose query encoder and search effort can be selected together for each request. Preserving the index therefore preserves the document asset while allowing the recurring computation to change.

For a researcher, the pooled teacher-quality comparison conceals structure that conditioning on dimensionality exposes. The prospective head result supports that predictive account, while the table result and unfavorable original-roster selections limit its practical reach. Cheap fitting supplies a direct alternative

to relying on the forecast. The recurring graph deficit extends the analysis from representation quality to retrieval computation, including a lookup path learned jointly with its documents.

These results have three consequences for a search system. First, a candidate query encoder is cheap to fit and to measure on development queries for the target index. On our original roster, that measurement selects better than original query quality, and the vector-agreement counterexamples show that cosine fidelity cannot certify the choice. Second, a replacement query path has its own search requirements. The original queries’ distance ratio flags susceptible spaces before any replacement is fitted; the size of the penalty on a new workload still needs measurement on that workload. Third, a compatible family turns query compute into a per-request setting over the same stored documents. Allocation by request context needs no predictor; predicting which encoder will retrieve best for a given query remains unresolved.

The selected own-index references also delimit the proposition. bge-small scores 0.5171 and LEAF 0.5402 on BEIR-15, above Nano’s 0.5081, using their own document indexes. Keeping one Stella index lets each request choose among three query budgets; separate models would need separate indexes and ingestion pipelines to offer those choices. The family makes the compute budget a request-level choice over a shared document asset.

8. Limitations

The quality study covers two closed-form recipes over related English spaces at three dimensionalities. The shared vocabulary constrains the table roster; transfer to other tokenizers, vendor-provided spaces, and fully trained students remains open. Dimensionality covaries with family and quality. Checkpoint intervals describe this roster, while family-held-out and prospective tests probe specific extensions.

The search study covers one engine, one graph configuration, and three corpora, including TREC-COVID’s 50 queries. Its geometry predictor forecasts recovery gaps, not their mechanism. Cross-space timing used a shared machine; latency conclusions use the separate Constella sweep. LightRetriever recurrence covers one released model and two corpora.

Constella’s training pools differ, including FEVER exposure; Stella discloses ArguAna, FiQA, and FEVER exposure. Quality intervals condition on fixed models. Timing covers warmed sequential CPU execution, while the million-passage diagnostic preserves positives. Request-context selection is a capability and an illustrative policy; concurrent operation and real typeahead quality were not measured.

Artifact availability

The [companion repository](#) contains inference examples, pinned configurations, dated methods and amendments, immutable result receipts, and the full research history, including secondary analyses and negative results. Released models are [Constella Zero](#), [Constella Nano](#), and the [Stella document encoder](#). The cleaned 337,981-query fit list and the one-million-passage identifier list are identified by hash but not yet archived. Prepared training corpora are described by source and file manifests.

Appendix A. Registered evaluations

Serving configuration. Stella is pinned to revision `ffeb2b7e`, with its `s2p_query` prompt, fp32 encoding, normalized output, fp16 stored document vectors, and cosine similarity; maximum sequence length 512. Zero’s training pool includes FEVER-train and HotpotQA; Nano excludes FEVER. MS MARCO is licensed for non-commercial use, so it was validation-only in our distillation: it supplied no training targets, negatives, or generation seeds, although Nano’s pretrained base has prior exposure to it.

Partitions. The six-set suite is NFCorpus, SCIDOCS, SciFact, TREC-COVID, FiQA, and ArguAna. “Clean-four” means the first four, which have no disclosed Stella overlap; it is not a causal contamination estimate. Four datasets were held out and evaluated once: FEVER, DBpedia-entity, and the CQADupStack android and english forums. Two other forums, physics and programmers, select table penalties and routing settings and were reused during model construction.

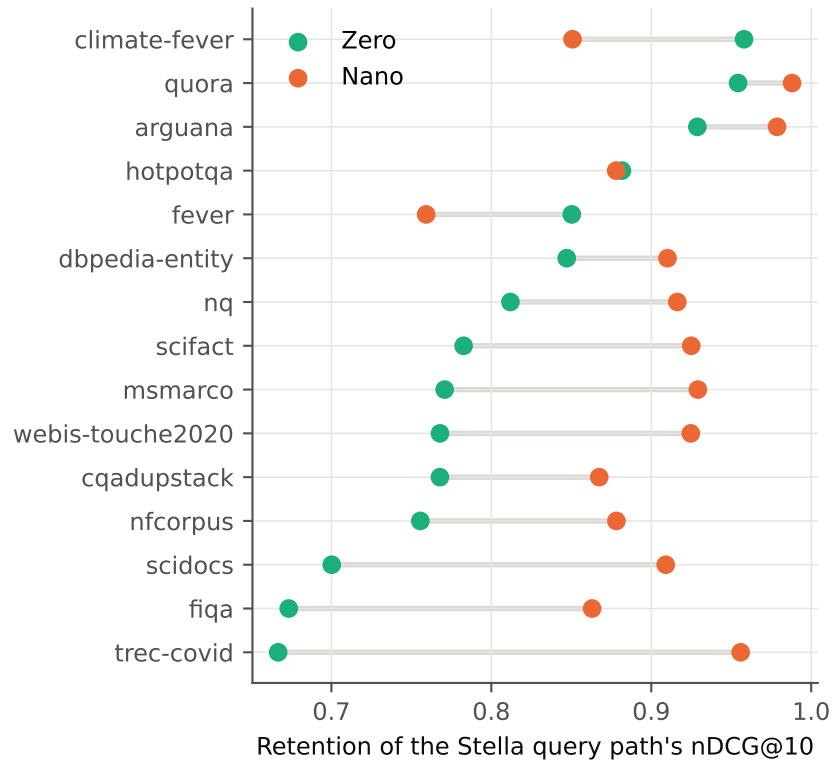


Figure A1: Retention per dataset. Datasets with disclosed Stella training exposure are ArguAna, FiQA, and FEVER; the four originally held-out sets are FEVER, DBpedia-entity, and the CQADupStack android and english forums.

Nano’s registered contrasts report nDCG@10 differences and used a fixed sequence with one-sided 2.5% lower bounds. “Established” means the bound cleared zero; “unresolved” means it did not; “descriptive” rows were registered as reports, not tests. Intervals resample queries with the models fixed.

Contrast	Partition	Score difference	Outcome
Nano – bge-small	clean-four	+0.0176; lower bound +0.0037	established
Nano – bge-small	all six	+0.0274; lower bound +0.0173	established
Nano – LEAF	all six	+0.0162; lower bound +0.0065	established
Nano – LEAF	clean-four	−0.0011; lower bound −0.0145	unresolved
Nano – bge-small	held-out	+0.0032 [−0.0069, +0.0134]	descriptive
	non-double-exposure aggregate, dataset weighted		
Nano – bge-small	held-out	−0.0121 [−0.0219, −0.0024]	descriptive
	non-double-exposure aggregate, query pooled		
Nano – LEAF	held-out	−0.0389 [−0.0488, −0.0290]	descriptive
	non-double-exposure aggregate		

The registered non-double-exposure aggregate uses DBpedia-entity weighted one half, the android and english forums one quarter each, FEVER excluded as the double-exposure sensitivity row.

Zero’s registered contrasts report nDCG@10 differences and used a Holm correction across the family. The dense comparison tests whether Zero exceeds LightRetriever’s dense path under per-task prompts, whose six-set nDCG@10 is 0.4583. Zero scores below that reference and does not confirmatorily beat BM25.

Contrast, all six	Score difference	Raw 95% interval	Outcome
Zero – LightRetriever dense	−0.0243	[−0.0405, −0.0086]	below LightRetriever
Zero – BM25	+0.0165	[+0.0017, +0.0311]	unresolved; p=.0149 fails Holm .0083
Zero + BM25 (convex) – OpenSearch	+0.0043	[−0.0063, +0.0151]	unresolved

The registered clean-four sensitivity is descriptive: −0.0443 [−0.0675, −0.0212], −0.0311 [−0.0517, −0.0109], and −0.0107 [−0.0262, +0.0043]. Unresolved superiority does not establish equivalence. The confirmatory fusion used convex score fusion with development-fitted weight 0.8 at depth 1000. Later distribution-based score fusion at 100 candidates is a deployability choice, not a new confirmation.

The held-out four, copied from their one-shot published aggregate (registered):

System	FEVER	DBpedia	Android	English
Nano	0.6231	0.4190	0.4774	0.4298
Zero	0.6978	0.3900	0.4431	0.3690
Stella	0.8207	0.4603	0.5275	0.5183
bge-small	0.8662	0.4003	0.4761	0.4558
LEAF	0.8671	0.4520	0.5260	0.4707
BM25	0.5036	0.3045	0.3952	0.3481
Zero + BM25	0.7559	0.4030	0.4629	0.4016
Nano + BM25	0.6879	0.4296	0.4750	0.4323

Precision. All released paths run in fp32 through the published loader.

Appendix B. Reproduction

Closed-form table. Let X contain token counts divided by query token count, Y the original query path’s vectors, and W_0 its normalized outputs for individual tokens encoded between special tokens. Fit

$$\min_W \|XW - Y\|_F^2 + \lambda \|W - W_0\|_F^2.$$

The anchor includes the original model’s pooling and output projection. Student tokenization includes special tokens, has no query prefix, and truncates at 512 tokens; mean pooling and L2 normalization follow lookup. Block conjugate gradient solves the normal equations, requiring every output column to pass relative-residual tolerance 1e-6 in fp32 or 1e-8 in fp64. Penalties range from 1e-4 to 1e-1, with one decade extension when development selection lands at an edge. Exact development nDCG@10 selects the penalty before six-set scoring. The fit list was cleaned after an earlier version had 1.31% held-out-query overlap. One expanded-roster checkpoint, arctic-embed-m-v1, stopped at the convergence gate and is excluded from the 26. Encoder exposure was documented for the registered roster; the exploratory expansion was not individually audited.

Closed-form head. Frozen bge-small revision 5c38ec7c supplies masked-mean states from layers 12, 8, and 4. Concatenating a constant bias feature yields F with 1,153 columns. A dense solve fits

$$\min_A \|FA - Y\|_F^2 + \lambda \frac{\text{tr}(F^\top F)}{1153} \|A\|_F^2.$$

The bias is regularized; predictions are normalized. Penalty selection matches the table. bge-small is itself a roster entry sharing this backbone. The original query path’s prompts apply to targets, while students have no query prefix.

Zero: data and distillation. The pool has 338,076 usable query-document pairs and 220,632 query-only rows from Amazon ESCI, FEVER-train, HotpotQA-train, SQuAD-train, Mr. TyDi English, NQ Open, and TriviaQA. Training sources permit commercial derived weights, with attribution where required. The distillation phase also uses 924,704 short document spans as pseudo-queries. Table rows start from Stella’s normalized token-in-context outputs; scalar weights start from inverse document frequency. Stella targets and document vectors stay frozen.

The first phase runs 16,000 steps, batch 512, with ordinary count pooling. Half the batch comes from query-only rows and pseudo-queries with cosine alignment loss. Paired queries combine cosine alignment and KL divergence against Stella’s score distribution over a positive and 31 distractors from a document bank; the extra-text cosine mean is added separately. Cosine and KL weights are 1, temperature 0.02. Adam learning rates are 3e-3 for rows and 1e-2 for weights. A row penalty of 1e-3 anchors initialization, divided by one plus the row’s update count.

Zero: ranking and export. The second phase runs 2,500 contrastive steps from that checkpoint. InfoNCE uses 32,768 bank negatives plus provided negatives, at temperature 0.02. Known positives and candidates scoring above the positive’s Stella score minus 0.02 are masked. Adam peaks at 1e-3 for rows and 1e-2 for weights, with 200 warmup steps and linear decay to a tenth. The row penalty anchors the phase’s starting checkpoint. Both phases use seed 0 and a bank of 2,000,000 document vectors.

Development selected the two-phase recipe, then square-root count pooling for its unchanged table. Export folds scalar weights into rows and applies per-row absolute-maximum int8 quantization with fp32 scales. A token appearing c times contributes $\sqrt{c}$ times its dequantized row before normalization. Unupdated rows retain initialization. Development comprises NQ, HotpotQA, two CQADupStack forums, and two training-source slices..

Nano: data and initialization. The deduplicated pool contains 3,486,034 query texts: 403,443 real queries from ESCI, HotpotQA, Mr. TyDi, NQ Open, SQuAD, and TriviaQA; 999,744 PAQ questions; 1,248,385 harvested texts; and 834,462 generated queries. Harvested text comes from Wikipedia, arXiv metadata, and the licensed document pool. Sampling gives equal presentation shares to 12 forms: factoid, how-to, claim, argument, finance, title, keyword, health, product, comparison, yes/no, and conversational. The 6,070,049 eligible documents replay in reshuffled epochs. FEVER is excluded. Queries have no student prefix; document examples use `passage::`; Stella supplies corresponding query or document targets.

Qwen3-8B-AWQ revision 4da05a8e generates seven forms with thinking disabled, temperature 0.8, top-p 0.95, and passage-derived seeds. Finance and health use Wikipedia passages; other forms use HotpotQA. Filtering applies length checks, exact and near-duplicate handling, document hold-outs, overlap screens, and copied-span rejection. Pinned prompts, sampling limits, filters, and source manifests remain in the companion repository.

The bge-small backbone uses revision 5c38ec7c. Its three-layer features project to 1,024 dimensions, then masked mean pooling and normalization. Ridge initialization uses 60,000 queries, seed 21, and a 50,000/10,000 split to select normalized squared-L2 error over penalties 1e-6 to 1; the selected 0.001 penalty is refitted on all queries. The affine projection commutes with mean pooling.

Nano: optimization and selection. Backbone and projection train against normalized Stella vectors with squared-L2 loss, alternating three query batches and one document batch. AdamW uses batch 32, seed 0, betas (0.9, 0.999), epsilon 1e-8, matrix weight decay 0.01, zero decay for one-dimensional parameters, and gradient clipping at norm 1. Forward passes use bf16; loss uses fp32. Three cycles decay from 1e-4 to 1e-5, with 64,000-example warmup in the first. The released third cycle-end checkpoint follows 199,999,721 example presentations. Registered screening and monitoring use an equally weighted four-family development macro: six BRIGHT slices, MedicalQARetrieval, LEDGER, and LegalBench CorporateLobbying and ConsumerContractsQA. Defaults remain when comparisons are unresolved. Full configuration, generation prompts, selection rules, and artifact hashes are retained in the repository.

Search protocol. Qdrant 1.19.1 uses cosine HNSW with $m=16$, $ef_construct=100$, indexing threshold 1 KB, and ef from 16 to 512. Cross-space graphs cover FiQA, SCIDOCS (25,657 documents), and TREC-COVID (171,332), uncompressed, with two seeded insertion-order builds per workload. Search requests 11 results before the registered self-hit drop. Recovery counts a returned document when its exact score reaches the exact 10th-neighbor score within cosine tolerance 1e-4. NumPy parity is checked on 200 queries per path; spaces below 0.995 are excluded. This removes tas-b, whose SCIDOCS head parity is 0.994. The arctic-embed-l mean-pooling control is outside roster statistics.

The smallest student ef reaching the original query path’s relative nDCG loss at $ef=64$, divided by 64, is computed per build and averaged. Recovery uses the analogous rule. Failure by $ef=512$ is censored, with multiplier at least eight. SCIDOCS addition, tie handling, parity threshold, and exclusion amendments preceded analysis of any space’s rows; dated methods retain their history. Gap-predictor intervals resample each space with all workload rows. The distance ratio uses a 5,000-document neighbor sample; family-held-out ridge prediction uses standardized features and unit penalty.

LightRetriever replication uses the released Qwen2.5-1.5B adapter and primary web-search instruction, comparing lookup and full query encoding over the same document vectors under the cross-space search protocol. The repository pins its revision and prompt.

Instance and timing workloads. FiQA uses all 57,638 documents and 648 queries. The MS MARCO diagnostic keeps every judged-positive passage for 6,980 development queries, then uniformly samples without replacement to 1,000,000 passages from the 8,841,823-passage corpus, seed 20260930. MS MARCO is validation-only, supplying no distillation targets, negatives, or generation seeds; Nano’s pretrained backbone has prior exposure.

The registered instance sweep checks exact parity against NumPy and measures warmed encoding and search separately, sequentially. Compressed sweeps use oversampling one, two, and four. The separate encoding sample contains 100 five-to-12-word queries, batch one, four CPU threads, Apple M5 Pro.

Appendix C. Encoder roster and prospective scores

Original roster. All scores are six-set exact nDCG@10. Pool is document-vector pooling: cls denotes the classification-token representation, and mean denotes mean pooling. Reg. marks the ten registered spaces; Expl. marks the exploratory expansion. Pinned revisions and prompts are in the companion configurations.

Encoder	Dim.	Pool	Original	Table	Head	Status
bge-small-en-v1.5	384	cls	0.5042	0.3581	0.5084	Expl.
arctic-embed-s	384	cls	0.4993	0.3516	0.2594	Expl.
gte-small	384	mean	0.4838	0.2903	0.4733	Expl.
arctic-embed-xs	384	cls	0.4662	0.3440	0.2779	Expl.
e5-small-v2	384	mean	0.4544	0.3203	0.3367	Expl.
minilm-l12	384	mean	0.4219	0.3138	0.3229	Expl.
minilm-l6	384	mean	0.4142	0.3267	0.3088	Expl.
multi-qa-minilm-l6	384	mean	0.4008	0.3406	0.3094	Expl.
msmarco-minilm-l6	384	mean	0.3281	0.2905	0.1962	Expl.
gte-base-en-v1.5	768	cls	0.5331	0.3252	0.2618	Reg.
arctic-embed-m-v1.5	768	cls	0.5263	0.3279	0.1874	Reg.
bge-base-en-v1.5	768	cls	0.5259	0.3529	0.3249	Reg.
bge-base-en-v1	768	cls	0.5131	0.3004	0.2773	Expl.
gte-base	768	mean	0.5063	0.2762	0.2314	Expl.
e5-base-v1	768	mean	0.4934	0.3126	0.2345	Expl.
e5-base-v2	768	mean	0.4669	0.2930	0.2241	Reg.
contriever-msmarco	768	mean	0.3909	0.3270	0.1879	Expl.
tas-b	768	cls	0.3262	0.2115	0.0868	Expl.
contriever	768	mean	0.2846	0.1864	0.0590	Expl.
gte-large-en-v1.5	1024	cls	0.5970	0.2455	0.2229	Reg.
stella-400M-v5	1024	mean	0.5745	0.3974	0.2505	Reg.
mxbai-embed-large-v1	1024	cls	0.5368	0.2605	0.2641	Reg.
bge-large-en-v1.5	1024	cls	0.5329	0.2845	0.2679	Reg.
arctic-embed-l	1024	cls	0.5289	0.3034	0.1564	Reg.
gte-large	1024	mean	0.5129	0.2481	0.2249	Expl.
e5-large-v2	1024	mean	0.4735	0.2585	0.1743	Reg.

Prospective roster (exploratory). Eight encoders were fixed before encoding. Predictions from the original 26 were committed by hash before student fitting, with each command verifying that hash. The e5-base-unsupervised table stopped at the convergence gate at penalty 1e-5; its completed head remains included. Each student column pairs actual and predicted six-set nDCG@10.

Encoder	Dim.	Original	Table, actual / predicted	Head, actual / predicted
bge-large-en v1	1024	0.5237	0.2613 / 0.2883	0.2481 / 0.2103
e5-large	1024	0.4914	0.2875 / 0.2757	0.2243 / 0.1825
nomic-embed-text-v1	768	0.4851	0.3684 / 0.2945	0.2418 / 0.2336
e5-base-unsupervised	768	0.4562	stopped / 0.2833	0.1178 / 0.2087
all-MiniLM-L12-v1	384	0.4115	0.3256 / 0.3171	0.3340 / 0.3062
msmarco-distilbert-base-v4	768	0.3436	0.3071 / 0.2395	0.1734 / 0.1117
msmarco-bert-base-dot-v5	768	0.3239	0.2127 / 0.2318	0.1032 / 0.0947
paraphrase-MiniLM-L6-v2	384	0.3001	0.2566 / 0.2738	0.2447 / 0.2103

References

- [BGE] BAAI. bge-small-en-v1.5. Model card and pretrained model. [Model repository](#).
- [QED] Yuxuan Wang and Hong Lyu. Query Encoder Distillation via Embedding Alignment is a Strong Baseline Method to Boost Dense Retriever Online Efficiency. SustainNLP Workshop at ACL, 2023. [arXiv:2306.11550](#).
- [EmbedDistill] Seungyeon Kim, Ankit Singh Rawat, Manzil Zaheer, et al. EmbedDistill: A Geometric Knowledge Distillation for Information Retrieval. 2023. [arXiv:2301.12005](#).
- [LEAF] Robin Vujanic and Thomas Ruecksties. LEAF: Knowledge Distillation of Text Embedding Models with Teacher-Aligned Representations. 2025, revised 2026. [arXiv:2509.12539](#).
- [LightRetriever] Guangyuan Ma, Yongliang Ma, Xuanrui Gou, Zhenpeng Su, Ming Zhou, and Songlin Hu. LightRetriever: A LLM-based Text Retrieval Architecture with Extremely Faster Query Inference. ICLR, 2026. [arXiv:2505.12260](#).
- [pyNIFE] Stephan Tulkens. pyNIFE: Nearly Inference Free Embeddings in Python. Software, 2025. [Repository](#).
- [Cho and Hariharan 2019] Jang Hyun Cho and Bharath Hariharan. On the Efficacy of Knowledge Distillation. ICCV, 2019. [Proceedings](#).
- [PROD] Zhenghao Lin, Yeyun Gong, Xiao Liu, et al. PROD: Progressive Distillation for Dense Retrieval. WWW, 2023. [arXiv:2209.13335](#).
- [HNSW] Yury A. Malkov and Dmitry A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE TPAMI 42(4), 2020. [arXiv:1603.09320](#).
- [OOD-DiskANN] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. 2022. [arXiv:2211.12850](#).
- [RoarGraph] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. PVLDB 17(11): 2735-2749, 2024. [Paper](#).
- [Arabzadeh et al.] Negar Arabzadeh, Xinyi Yan, and Charles L. A. Clarke. Predicting Efficiency/Effectiveness Trade-offs for Dense vs. Sparse Retrieval Strategy Selection. CIKM, 2021. [arXiv:2109.10739](#).
- [QPP] Guglielmo Faggioli, Thibault Formal, Stefano Marchesin, Stéphane Clinchant, Nicola Ferro, and Benjamin Piwowarski. Query Performance Prediction for Neural IR: Are We There Yet? 2023. [arXiv:2302.09947](#).
- [DBSF] Qdrant contributors. Hybrid queries: distribution-based score fusion. Qdrant documentation. [Page](#).
- [Qdrant 1.19.1] Qdrant contributors. Binary query encoding and vector-index search implementation, v1.19.1. [Tagged source](#).
- [Stella] Stella-en-400M-v5. Model card for NovaSearch/stella_en_400M_v5. [Model page](#).
- [BEIR] BEIR contributors. BEIR retrieval benchmark, datasets, and evaluation software. [Repository](#).
- [Index migration] Qdrant contributors. Migrate to a New Embedding Model with Zero Downtime in Qdrant. Qdrant documentation. [Migration procedures](#).